

Construire une application intégrant l'IA

Call To Adventure · Bloc 3 · E4 : 20 minutes

PRODUIT

Une aventure interactive dans le navigateur, avec narration IA et règles de jeu côté serveur.

RÉALISATION

Cadrer, concevoir, organiser et développer les parcours du joueur.

QUALITÉ / LIVRAISON

Tester le métier et les accès.
Automatiser les contrôles.
Préparer un artefact déployable.

Le bloc 3 relie les besoins du joueur aux preuves de fonctionnement.

Permettre de jouer et de retrouver sa progression

Les parcours distinguent le visiteur et le joueur connecté.

VISITEUR

- Choisir une aventure.
- Créer un personnage valide.
- Jouer sans créer de compte.
- Progression liée à la session.

JOUEUR CONNECTÉ

- Retrouver ses sauvegardes.
- Reprendre une partie active.
- Réutiliser ses personnages.
- Gérer ses propres données.

EXPLOITANT / JURY

- Installer et vérifier le service.
- Comprendre les choix techniques.
- Relier exigences, code, tests et démonstration.

La continuité de la partie et la propriété des données orientent la conception.

Relier les demandes à des réalisations concrètes

Jeu : critères formalisés ; alternance : évolutions de plateforme réalisées.

CALL TO ADVENTURE

Reprise des sauvegardes.
Propriété des données.
Échec IA sans progression.
Critères et tests disponibles.

DATAPLATEFORME

Demandes de fonctionnalités
prises en charge.
Bugs corrigés et adaptation à la
charge.

PREUVE À JOINDRE

Une demande réelle, sa
reformulation, le changement livré
et sa validation par l'utilisateur
métier.

La plateforme illustre la relation au besoin ; le contexte applicatif IA reste explicite.

Séparer présentation, métier et persistance

Une application Django avec des services Python internes.

PRÉSENTATION

Templates HTML / CSS / JS



Vues Django et endpoints JSON



Sessions, auth et CSRF

MÉTIER

GameEngine



LangGraph Combat

IA + RAG règles serveur

STOCKAGE / SERVICES

Base Django : comptes / parties.

SQLite catalogue : références.

Chroma / Ollama : contexte.

Fournisseur externe : génération.

Le navigateur envoie des actions ; le serveur décide des mutations autorisées.

Choisir une architecture adaptée au périmètre

Les choix actuels favorisent une démonstration locale reproductible.

DJANGO

Comptes, sessions, ORM et templates dans un même cadre.
Moins de couches à exploiter pour ce projet solo.

SQLITE / SERVICES IA

Données locales de taille modeste.
IA hébergée et retrieval local.
Limites de concurrence et de dépendance externe documentées.

CIBLE DE DÉPLOIEMENT

Conteneur web et proxy HTTPS.
Secrets externes, stockage durable.
Hébergement et SGBD de production à confirmer.

La préproduction cible est décrite ; son accessibilité réelle reste à prouver.

Porter une réalisation de bout en bout

Alternance : développement, maintenance et évolution de la plateforme.

RÉALISER

Chaîne multi-source complète.
Airflow, ClickHouse et dbt.
Documentation des travaux.

ADAPTER

Scalabilité.
Résolution des bugs.
Demandes de fonctionnalités.
Arbitrages à illustrer.

COORDONNER

Préciser interlocuteurs, priorités,
outils de suivi, revues et échanges
réellement pratiqués.

L'autonomie technique renforce le récit ; les pratiques de coordination doivent être prouvées.

Conserver une progression cohérente

Le moteur expose des transitions de jeu et des règles de persistance.

PARCOURS

Accueil → personnage → récit
↓
Choix → combat éventuel
↓
Victoire, défaite ou poursuite

ÉTAT

Anonyme : session Django.
Connecté : SaveGame + session.
État sérialisé en JSON.
Partie terminée non rechargeable.

RÈGLES MÉTIER

Combat par session.
Soins et dégâts côté serveur.
Propriété des sauvegardes.
Échec IA sans progression persistée.

Les tests portent sur les transitions et les droits, au-delà du rendu des pages.

Protéger les requêtes et les données du joueur

Des contrôles concrets, avec des limites de production identifiées.

REQUÊTES

CSRF sur les POST navigateur.
Taille JSON limitée.
Quota sur les routes IA.
Contenu dynamique via
textContent.

PROPRIÉTÉ / SECRETS

Filtres par utilisateur.
État de combat isolé.
Secret et hôtes de production via
environnement.

LIMITES

Audit dépendances à compléter.
Cache partagé pour plusieurs
instances.
Revue des appels longs et de la
concurrence du graphe.

Les protections du navigateur ont évolué : le code courant prime sur les anciennes notes.

Rendre les interactions perceptibles et maîtrisées

Améliorations présentes ; validation manuelle encore nécessaire.

INTERACTIONS

Annonces de statut dynamiques.
Onglets avec sémantique ARIA.
Focus après mise à jour.
Barres de vie exposées comme progression.

VÉRIFICATION MANUELLE

Parcours au clavier.
Contrastes et zoom 200 %.
Lecteur d'écran.
Qualité des textes FR / EN.

SOBRIÉTÉ

Appels de jeu déclenchés par action.
Réutilisation du retrieval.
Images locales.
Poids et consommation à mesurer.

La présence d'attributs ARIA ne suffit pas à valider l'accessibilité réelle.

44 tests ciblés réussis sur le bloc applicatif

Vérifications locales du 14 septembre 2026.

LOT EXÉCUTÉ

Combat et isolation.
Sécurité des requêtes.
Templates et parcours simulé.
Sauvegardes, personnages,
métriques.

MIGRATIONS

`makemigrations --check`
`--dry-run`

Aucun changement détecté.
Aucune migration créée.

CI VERSIONNÉE

PR, push main / master, manuel.
Lint, checks, tests et couverture.
Dépendances verrouillées.
Résultats distants à joindre.

44 tests ne désignent ni la suite complète ni un pourcentage de couverture.

Construire, vérifier et conserver le même artefact

Le conteneur constitue l'unité de livraison prévue.

BUILD / SMOKE

Dockerfile Python 3.12.
Installation verrouillée.
Démarrage Gunicorn.
Image taguée SHA, test /health
puis export tar en CI.

EXPLOITATION

Configuration par environnement.
Données persistantes à prévoir.
Migrations au démarrage local.
Étape de release à contrôler en
production.

PROMOTION / RETOUR

Déployer le digest vérifié.
Conserver l'image précédente.
Contrôler la compatibilité DB.
Rejouer santé et parcours.

Le job staging est un placeholder : il ne déploie pas encore l'application.

Montrer le parcours, puis les garanties

Séquence préparée : 1 min 40, dans un environnement de démonstration.

00:00 → 00:35

Ouvrir une aventure et un personnage déjà prêts.
Afficher récit et choix.
Exécuter une action de jeu.

00:35 → 01:10

Montrer une sauvegarde du compte de test et sa reprise.
Présenter le résultat du test de propriété.

01:10 → 01:40

Afficher les résultats locaux et le workflow.
Nommer les preuves manquantes.
Introduire le cas d'incident E5.

Prévoir une capture de secours ; annoncer toute simulation utilisée.

Fiabiliser l'ingestion ServiceNow vers ClickHouse

Incident décrit par le candidat ; pièces techniques de résolution à joindre.

SYMPTÔME

Erreurs répétées de validation de types pendant l'ingestion des données ServiceNow.

CAUSE IDENTIFIÉE

Dans ce flux, les lignes issues de XML apportent des chaînes brutes non adaptées aux types attendus.

CORRECTION

Développement d'un système de validation et de conversion vers les types de données attendus.

Cas data professionnel : son adéquation au contexte applicatif IA d'E5 reste à établir.

Identifier le signal qui révèle l'échec d'ingestion

Le dispositif réel de détection de l'entreprise reste à préciser.

OBSERVATION

Bugs de validation de types constatés pendant l'ingestion.
Joindre un message d'erreur réel et anonymisé.

CONTEXTE

Table concernée.
Étape du DAG.
Champ et type attendus.
Valeur ou forme d'entrée impliquée.

DÉTECTION

Préciser comment le défaut a été détecté : run, logs, alerte ou signalement.
Ne pas inventer d'alerte.

La présence d'Airflow ne prouve pas une chaîne d'alerting opérationnelle.

Suivre le succès et la qualité de l'alimentation

Signaux à documenter ou à ajouter ; pas de seuils entreprise fournis.

EXÉCUTION

État et durée du DAG.
Retard de la donnée.
Fenêtre réellement traitée.
Reprises éventuelles.

QUALITÉ

Échecs de conversion.
Lignes acceptées / invalides.
Champs et colonnes nouveaux.
Écarts de schéma.

RÉACTION

Seuils et destinataire réels.
Procédure de diagnostic.
Preuve d'une détection puis d'une résolution.

Ces contrôles sont proposés ; leur présence et leurs valeurs ne sont pas affirmées.

Isoler une ligne qui déclenche l'erreur de type

Construire la preuve à partir du cas réel rencontré.

ENTRÉE RÉELLE

Un extrait XML anonymisé.
Une valeur brute problématique.
Le champ concerné.

ATTENDU

Le type prévu dans ClickHouse.
La règle de conversion.
La politique pour une valeur
invalidé.

OBSERVÉ AVANT

Message d'erreur de type.
Étape et commande d'ingestion.
Échantillon minimal reproductible.

La reproduction n'a pas encore été exécutée dans ce dossier : la pièce réelle reste à joindre.

Le format de transport ne porte pas les bons types

Le flux ServiceNow fournissait des chaînes issues de XML.

SOURCE

Dans l'intégration concernée :
données représentées en XML,
lues comme chaînes brutes.

ÉCART

Les valeurs transmises ne
correspondaient pas aux types
exigés par le schéma ClickHouse.

EFFET

L'ingestion rencontrait des erreurs
de validation.
Le contrôle devait intervenir avant
l'écriture typée.

Cette cause est rapportée par le candidat ; un exemple de valeur doit l'illustrer.

Valider et convertir avant de charger

Système de conversion des chaînes développé par le candidat.

ENTRÉE

Lire les valeurs brutes du flux
ServiceNow.
Identifier les types attendus selon
le contrat réel.

CONVERSION

Appliquer les règles développées
pour obtenir les valeurs typées
attendues.
Code à joindre.

VALIDATION

Contrôler avant l'insertion.
Documenter les cas invalides et le
comportement effectivement
choisi.

Ne pas remplacer silencieusement une donnée invalide par une valeur arbitraire.

Comparer le même échantillon avant et après

L'incident et sa correction sont décrits ; les résultats de validation restent à joindre.

AVANT

Erreur de type sur le cas identifié.
Entrée et schéma connus.
Trace du défaut.

APRÈS

Même entrée convertie.
Valeur typée obtenue.
Résultat ClickHouse à montrer.
Absence de perte à vérifier.

NON-RÉGRESSION

Cas valides et invalides.
Formats limites réellement rencontrés.
Test automatisé ou contrôle de reprise disponible.

Aucun taux de réussite ni compteur de lignes corrigées n'est inventé.

Une résolution métier à soutenir par ses pièces

La contribution est claire ; le dossier de preuve reste à compléter.

CONFIRMÉ

Incident de typage ServiceNow.
Investigation et cause.
Validation / conversion
développées.
Responsabilité de maintenance.

À JOINDRE

Message et échantillon
anonymisés.
Code / commit.
Avant / après.
Non-régression et trace de
livraison.

PÉRIMÈTRE E5

Relier le cas au contexte exigé.
Documenter la surveillance réelle.
Alternative disponible : incident de
combat du jeu.

Le cas entreprise ne remplace définitivement le cas du jeu qu'avec un périmètre et des preuves adaptés.

Le cas du combat reste disponible

Call To Adventure : défaut historique reproduit et correction vérifiée.

DÉFAUT

Deux combats peuvent partager les références globales dans l'ancien cœur de combat.

CORRECTION

Session de combat passée explicitement aux fonctions.
Commit historique identifié.

PREUVE

Script et JSON avant / après.
Test d'isolation.
Dossier E5 initial conservé dans presentation-bloc3/.

Ces résultats concernent le jeu ; ils ne valident pas le convertisseur ServiceNow.

Retrouver les pièces du dossier

Documentation applicative sous docs/block3/.

CADRAGE / RÉALISATION

C14 : besoins et traçabilité.
C15 : modèles et décisions.
C16 : backlog et revues.
C17 : code, qualité et sécurité.

QUALITÉ / LIVRAISON

C18 : tests et workflow CI.
C19 : Dockerfile, artefact et
procédure de promotion.
Preuves distantes à ajouter.

EXPLOITATION

C20 : monitoring, règles et logs.
C21 : dossier d'incident, correctif
Git et reproduction.
Ticket et déploiement à ajouter.

Les statuts des anciennes notes peuvent être en retard sur le code : dater les preuves.

Les points qui changent la solidité du dossier

Faire porter chaque réalisation sur son apport réel.

DATAPLATEFORME

Volumes importants par table.
Fenêtres dynamiques.
Colonnes ajoutées à la volée.
Conversion du XML ServiceNow.

BENCHMARKS

Un seul modèle exécuté.
Runner agnostique.
Scores et charge mesurés.
Référence Qwen à confirmer.

TINES

Quatre endpoints par webhook.
Contrat et appels EDR.
Intégration comme outils de l'agent Django.

Documents et runs à joindre ; contexte et responsabilité explicités pour chaque preuve.